

Linear Regression in R: A beginner's guide

Spring Semester 2026

This guide was inspired by the course material of Prof. Loukia Meligkotsidou for Linear Models and the R lab material by Dr. Ioannis Oikonomidis, Department of Mathematics, National and Kapodistrian University of Athens. All errors are my own.

Contents

Introduction	3
Getting Started with R	5
1 Data Validation & Visualisation	7
1.1 Qualitative Variables	8
1.2 Quantitative Variables	9
1.3 Normality Check	12
2 Fitting a Simple Linear Regression Model	14
2.1 Step 1: Fit the Models	14
2.2 Step 2: Interpret the Coefficients	17
2.3 Step 3: Hypothesis Testing	18
2.4 Step 4: Confidence Intervals for Coefficients (90%)	18
2.5 Step 5: Compare R^2 and Adjusted R^2	18
3 Fitting a Multiple Linear Regression Model	20
3.1 Step 1: Null Model and Full Model	20
3.2 Step 2: Model Selection Using Stepwise Regression	21
3.3 Model Selection: A Deeper Look	23
3.4 Step 3: The Best Fitted Model (AIC-based)	25
3.5 Step 4: Interpreting the Best Model Output	26
3.6 Step 5: Confidence Intervals for Coefficients (95%)	26
3.7 Step 6: Residual Diagnostics	27
3.7.1 Homoscedasticity Check	27
3.7.2 Normality Check	28
4 Analysis of Variance (ANOVA)	30
4.1 Step 1: Prepare the Data	30
4.2 Step 2: Two-Way ANOVA with Interaction	31

4.3	Step 3: Two-Way ANOVA without Interaction	31
4.4	Step 4: One-Way ANOVA	32
4.5	Step 5: Post-Hoc Comparisons (Tukey HSD)	32
4.6	Step 6: Residual Diagnostics	32

Introduction

“All models are wrong, but some are useful”
— George E. P. Box

This report serves as an introduction to linear regression and analysis of variance (ANOVA) using R, specifically designed for readers with little to no prior experience in statistical computing. The primary goal is not to exhaustively cover every theoretical detail, but rather to build practical intuition through hands-on examples, clear code, and meaningful interpretation of results.

Who is this for?

This guide is intended for readers who:

- Have basic knowledge of statistics (mean, variance, hypothesis testing)
- Want to learn how to implement linear models in R
- Seek to understand *what* the output means and *how* to validate it

What to expect

Throughout this report, we will:

1. Load, inspect, and validate real-world datasets
2. Fit simple and multiple linear regression models
3. Diagnose model assumptions using residual plots and statistical tests
4. Perform model selection using stepwise methods and information criteria (AIC, BIC)
5. Conduct one-way and two-way ANOVA with interaction effects
6. Interpret every number that appears in R’s output

A note on datasets

The datasets used in this report are deliberately clean and well-structured for educational purposes. Real-world data is rarely this tidy. However, mastering these fundamental techniques on clean data provides the necessary foundation to handle missing values, outliers, and violations of assumptions when they inevitably appear in practice.

Rationale

Black-box implementation of R functions is discouraged throughout this guide. Instead, we emphasize:

- Checking assumptions before trusting conclusions
- Understanding what each line of code actually does
- Visualizing data and residuals whenever possible
- Questioning results rather than accepting them blindly

As George Box famously reminded us, all models are indeed wrong. But with proper validation, careful interpretation, and healthy skepticism, some models become genuinely useful.

All code presented in this report is fully reproducible. Readers are strongly encouraged to run each example themselves and experiment with modifications.

Getting Started with R

Installing R and RStudio

Step 1: Install R

1. Go to <https://cran.r-project.org/>
2. Choose your operating system (Windows, Mac, or Linux)
3. Download the latest version (e.g., “Download R for Windows”)
4. Run the installer with default settings

Step 2: Install RStudio (Recommended)

R is the engine; RStudio is the dashboard. While R works on its own, RStudio makes everything easier.

1. Go to <https://posit.co/download/rstudio-desktop/>
2. Download the free RStudio Desktop version
3. Install after R is already installed

Tip: Install R first, then RStudio. If you do it backwards, RStudio won’t find R.

Project Organization:

Creating a project folder is recommended in order to work in a systematic way and keep track of projects.

The Working Directory

Your working directory is the folder R is currently looking at. When you ask R to read a file, it searches there first.

```
# Check current working directory
getwd()

# Set working directory (HARD-CODED - NOT RECOMMENDED)
setwd("C:/Users/YourName/My_R_Project") # Avoid this!
```

Dynamic Paths: The Better Way

Hard-coded paths like `C:/Users/Vasiliki/...` will break on your friend's computer. Use dynamic paths instead:

```
# Prefer: RStudio Project File (BEST)  
# Create a .Rproj file via File > New Project > Existing Directory  
# When you open the .Rproj file, the working directory automatically sets itself
```

Loading Data the Right Way

Data in project folder (Strongly Recommended)

Place your data file inside the `data/` folder, then:

```
library(here)  
df <- read.table(here("data", "prestige.txt"), header = TRUE)
```

First Look at Your Data

Once loaded, always inspect immediately:

```
# View first few rows  
head(df)  
  
# Summary statistics for each column  
summary(df)  
  
# Check for missing values  
colSums(is.na(df))
```

Recommended R Packages for This Guide

Install these packages before starting:

```
# Install packages (do once)  
install.packages(c("lmtest", "car", "MASS"))  
  
# Load packages (do each session)  
library(lmtest)      # Diagnostic tests (bptest, dwtest)  
library(car)  
library(MASS)        # Stepwise regression (stepAIC)
```

Remember: Reproducibility is a necessity. Organize your projects as if someone else, including future you, will need to run your code!

Chapter 1

Data Validation & Visualisation

Loading the Data

First, load the built-in `mtcars` dataset and inspect it:

```
# Load the built-in dataset
data(mtcars)

# Trim the dataset for simplicity (remove unnecessary columns)
cars <- mtcars[, -c(5, 7, 8, 10, 11)]

# First 6 rows
head(cars)

##           mpg cyl disp  hp   wt  am
## Mazda RX4    21.0   6  160 110 2.620  1
## Mazda RX4 Wag 21.0   6  160 110 2.875  1
## Datsun 710    22.8   4  108  93 2.320  1
## Hornet 4 Drive 21.4   6  258 110 3.215  0
## Hornet Sportabout 18.7   8  360 175 3.440  0
## Valiant      18.1   6  225 105 3.460  0
```

Missing Values Check

Always check for missing values before any analysis:

```
# Missing values per column
colSums(is.na(cars))

##  mpg  cyl disp  hp   wt   am
##    0    0   0   0    0    0

# Which rows have missing values
which(is.na(cars))
```



```
## integer(0)
```

The `mtcars` dataset contains no missing values, making it ideal for beginners.

Variable Selection

Select individual variables using the `$` symbol:

```
mpg <- mtcars$mpg   # Miles per gallon (response variable)
cyl <- mtcars$cyl    # Number of cylinders
disp <- mtcars$disp  # Displacement (cu.in.)
hp <- mtcars$hp      # Gross horsepower
wt <- mtcars$wt      # Weight (1000 lbs)
am <- mtcars$am      # Transmission (0 = auto, 1 = manual)
```

Note: `mpg`, `disp`, `hp`, `wt` are quantitative (continuous) variables, while `cyl` and `am` are qualitative (categorical) variables. Different statistics are reported in each case. For quantitative variables we usually report the mean and variance and provide histograms. For categorical variables, the median, quantiles, frequency tables and bar plots.

1.1 Qualitative Variables

Frequency Tables

```
# Frequency table for cylinders
cyl_freq <- table(cyl)
cyl_freq

## cyl
##  4  6  8
## 11  7 14

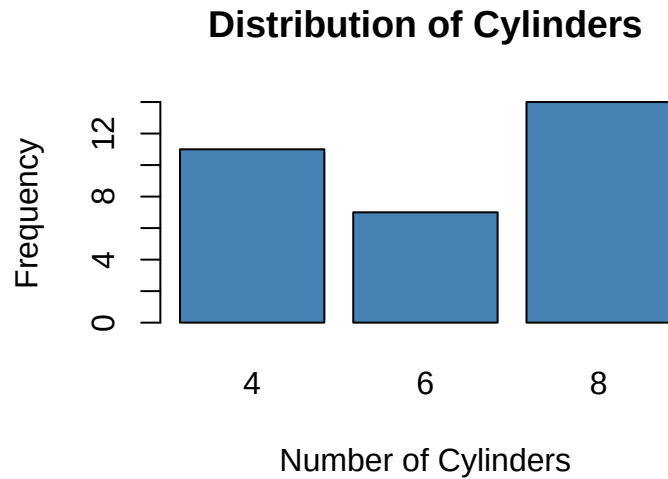
# Frequency table for transmission type
am_freq <- table(am)
am_freq

## am
##  0  1
## 19 13
```

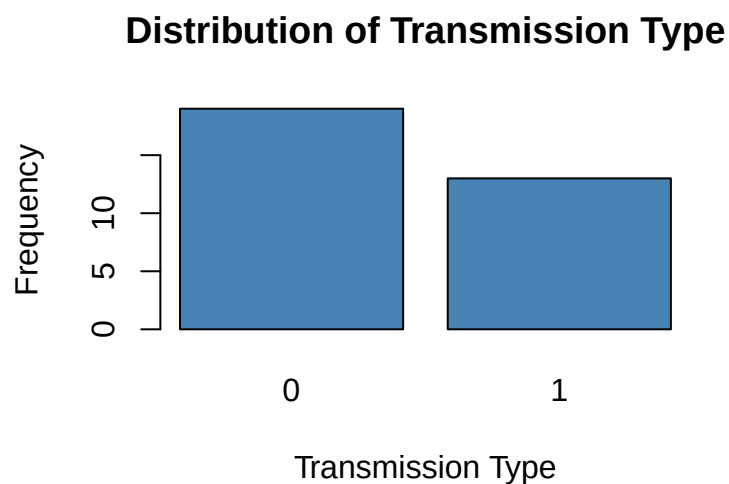
Bar Plots

```
# Bar plot for cylinders
barplot(cyl_freq,
        main = "Distribution of Cylinders",
```

```
xlab = "Number of Cylinders",
ylab = "Frequency",
col = "steelblue")
```



```
# Bar plot for transmission type
barplot(am_freq,
        main = "Distribution of Transmission Type",
        xlab = "Transmission Type",
        ylab = "Frequency",
        col = "steelblue")
```



1.2 Quantitative Variables

Summary Statistics

```

# Means
mean(mpg)

## [1] 20.09062

mean(displ)

## [1] 230.7219

mean(hp)

## [1] 146.6875

mean(wt)

## [1] 3.21725

# Or use colMeans for multiple columns
colMeans(cars[, 1:4])

##      mpg      cyl      displ      hp
## 20.09062  6.18750 230.72188 146.68750

```

Histograms with Density Curves

```

# Divide display into 4 columns for side-by-side plots
par(mfrow = c(1, 4))

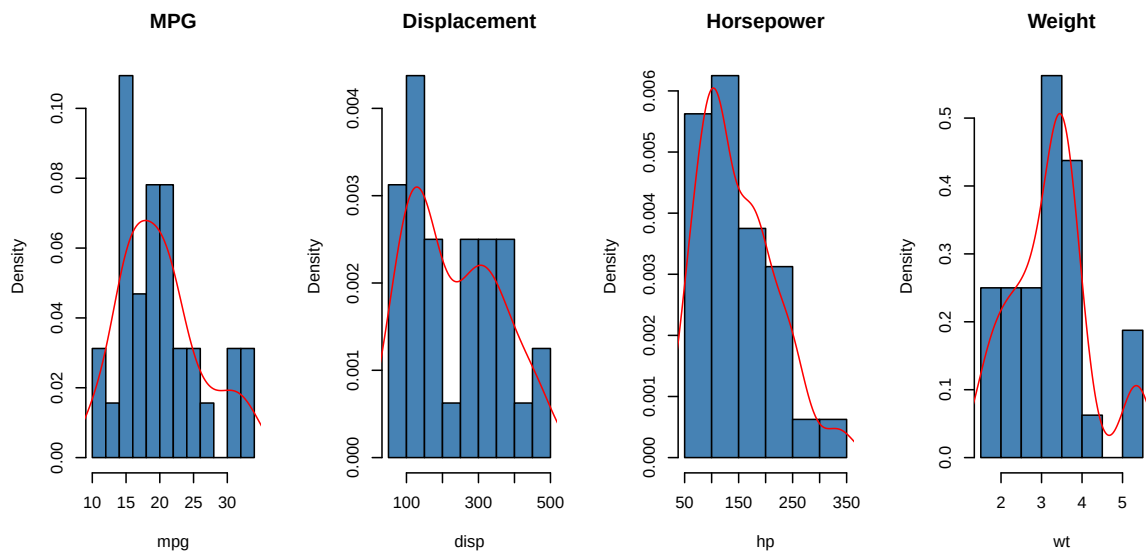
# Histogram for MPG
hist(mpg,
     main = "MPG",
     breaks = 10,
     col = "steelblue",
     freq = FALSE)
lines(density(mpg), col = "red")

# Histogram for Displacement
hist(displ,
     main = "Displacement",
     breaks = 10,
     col = "steelblue",
     freq = FALSE)
lines(density(displ), col = "red")

```

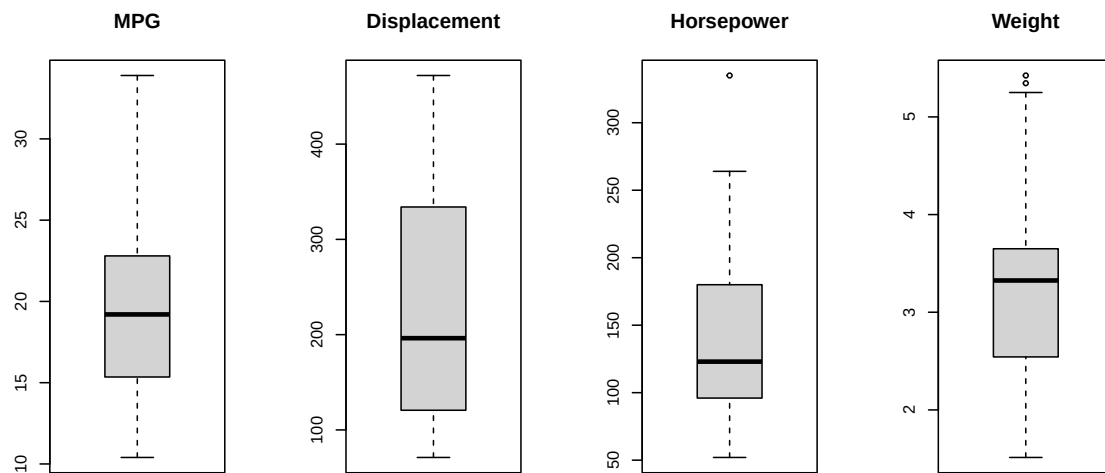
```
# Histogram for Horsepower
hist(hp,
     main = "Horsepower",
     breaks = 10,
     col = "steelblue",
     freq = FALSE)
lines(density(hp), col = "red")

# Histogram for Weight
hist(wt,
     main = "Weight",
     breaks = 10,
     col = "steelblue",
     freq = FALSE)
lines(density(wt), col = "red")
```



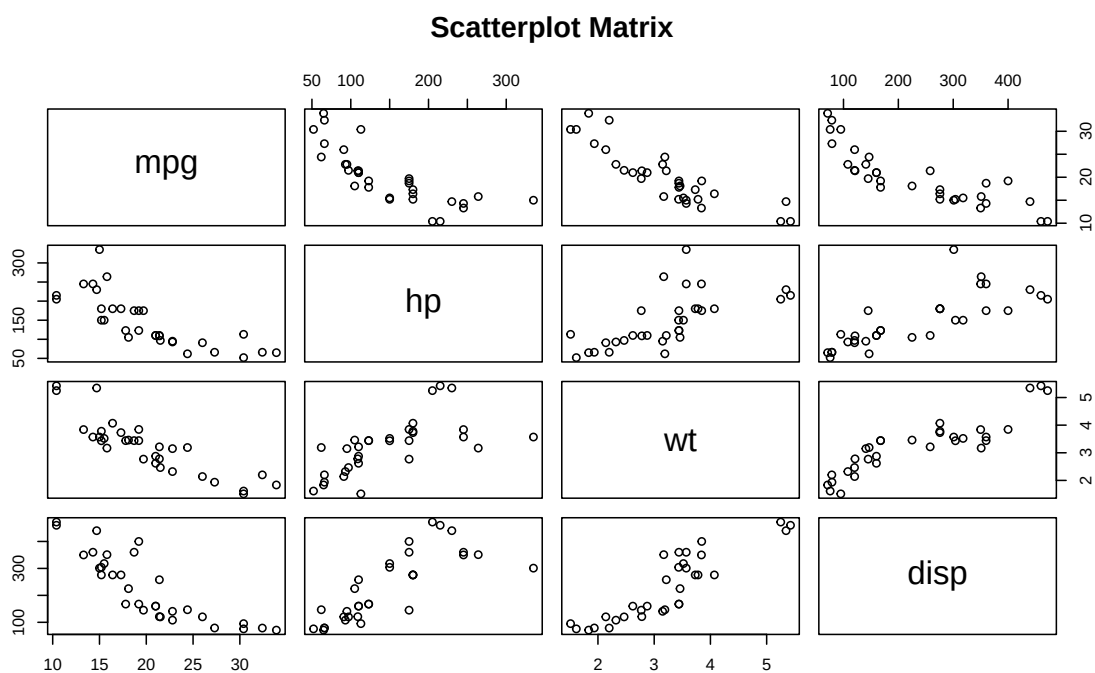
Boxplots

```
# Boxplots for outlier detection
par(mfrow = c(1, 4))
boxplot(mpg, main = "MPG")
boxplot(displacement, main = "Displacement")
boxplot(hp, main = "Horsepower")
boxplot(wt, main = "Weight")
```



Scatterplot Matrix

```
# Pairwise relationships between variables
pairs(cars[, c("mpg", "hp", "wt", "disp")],
      main = "Scatterplot Matrix")
```



1.3 Normality Check

The Shapiro-Wilk test examines whether data comes from a normal distribution.

```

# Test for MPG (Y variable)
shapiro.test(mpg)

##
## Shapiro-Wilk normality test
##
## data:  mpg
## W = 0.94756, p-value = 0.1229

# Test for Displacement
shapiro.test(displ)

##
## Shapiro-Wilk normality test
##
## data:  displ
## W = 0.92001, p-value = 0.02081

# Test for Horsepower
shapiro.test(hp)

##
## Shapiro-Wilk normality test
##
## data:  hp
## W = 0.93342, p-value = 0.04881

# Test for Weight
shapiro.test(wt)

##
## Shapiro-Wilk normality test
##
## data:  wt
## W = 0.94326, p-value = 0.09265

```

Interpretation of Shapiro-Wilk Test

If $p\text{-value} > 0.05$: Fail to reject $H_0 \rightarrow$ Data is normally distributed

If $p\text{-value} < 0.05$: Reject $H_0 \rightarrow$ Data is not normally distributed

Chapter 2

Fitting a Simple Linear Regression Model

We now fit two simple linear regression models using `mpg` as the dependent variable (Y):

1. **Model 1:** `mpg ~ wt` (predicting MPG using weight)
2. **Model 2:** `mpg ~ hp` (predicting MPG using horsepower)

2.1 Step 1: Fit the Models

Model 1: MPG ~ Weight

```
# Fit linear model for wt
m1 <- lm(mpg ~ wt)

# View results
summary(m1)

##
## Call:
## lm(formula = mpg ~ wt)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.5432 -2.3647 -0.1252  1.4096  6.8727
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  37.2851     1.8776   19.858 < 2e-16 ***
## wt          -5.3445     0.5591   -9.559 1.29e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

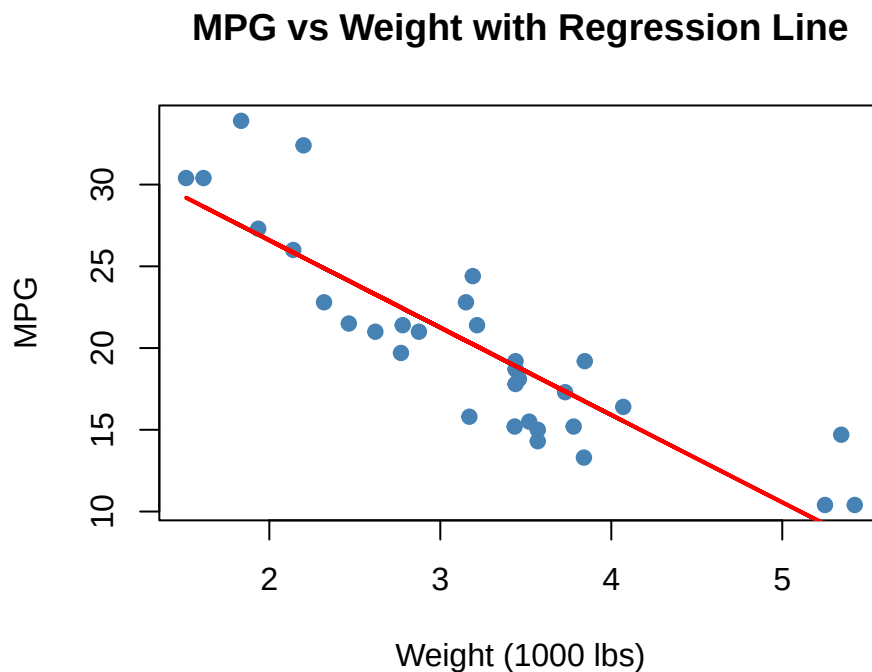
```
##  
## Residual standard error: 3.046 on 30 degrees of freedom  
## Multiple R-squared:  0.7528, Adjusted R-squared:  0.7446  
## F-statistic: 91.38 on 1 and 30 DF,  p-value: 1.294e-10
```

Extract Model Components

```
# Extract model components  
bhat_1 <- m1$coefficients      # Estimated coefficients  
ehat_1 <- m1$residuals        # Residuals (errors)  
yhat_1 <- m1$fitted.values     # Fitted (predicted) values  
stdehat_1 <- rstandard(m1)    # Standardized residuals
```

Scatter Plot with Regression Line

```
# Scatter plot with regression line  
plot(wt, mpg,  
      main = "MPG vs Weight with Regression Line",  
      xlab = "Weight (1000 lbs)",  
      ylab = "MPG",  
      pch = 19,  
      col = "steelblue")  
lines(wt, yhat_1, col = "red", lwd = 2)
```



Model 2: MPG ~ Horsepower

```
# Fit linear model for hp
m2 <- lm(mpg ~ hp)

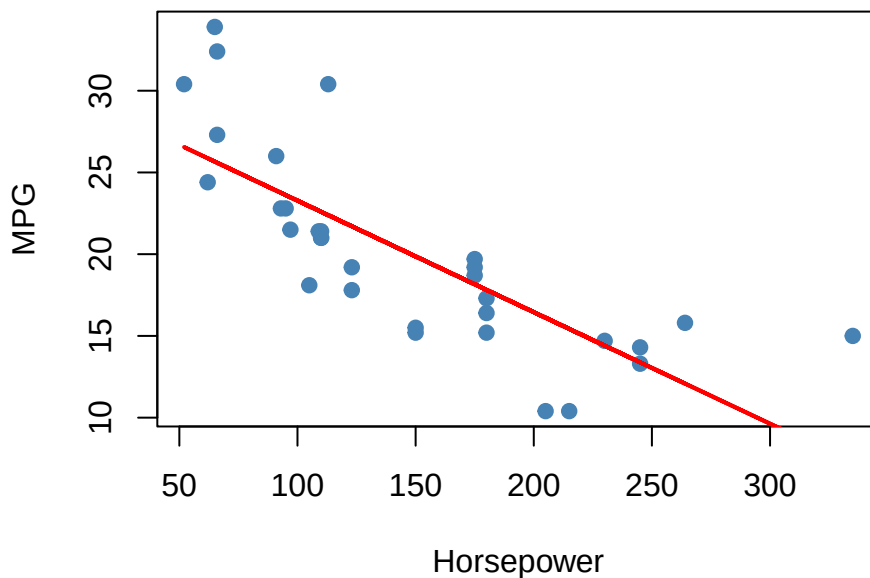
# View results
summary(m2)

##
## Call:
## lm(formula = mpg ~ hp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.7121 -2.1122 -0.8854  1.5819  8.2360
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 30.09886    1.63392   18.421  < 2e-16 ***
## hp          -0.06823    0.01012   -6.742 1.79e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.863 on 30 degrees of freedom
## Multiple R-squared:  0.6024, Adjusted R-squared:  0.5892
## F-statistic: 45.46 on 1 and 30 DF,  p-value: 1.788e-07
```

```
# Extract model components
bhat_2 <- m2$coefficients      # Estimated coefficients
ehat_2 <- m2$residuals        # Residuals (errors)
yhat_2 <- m2$fitted.values    # Fitted (predicted) values
stdehat_2 <- rstandard(m2)    # Standardized residuals
```

```
# Scatter plot with regression line
plot(hp, mpg,
      main = "MPG vs Horsepower with Regression Line",
      xlab = "Horsepower",
      ylab = "MPG",
      pch = 19,
      col = "steelblue")
lines(hp, yhat_2, col = "red", lwd = 2)
```

MPG vs Horsepower with Regression Line



2.2 Step 2: Interpret the Coefficients

Model 1 Interpretation (Weight)

- Intercept (37.2851): When weight is 0 lbs, the predicted MPG would be 37.29. (This is not meaningful in reality, as no car weighs 0 lbs.)
- Slope for wt (-5.3445): For each additional 1000 lbs of weight, MPG decreases by approximately 5.34 units.
- Statistical significance: p-value = 1.29e-10 (< 0.05) $\rightarrow$ weight is a statistically significant predictor of MPG.

Model 2 Interpretation (Horsepower)

- Intercept (30.0989): When horsepower is 0, the predicted MPG would be 30.10.
- Slope for hp (-0.0682): For each additional horsepower, MPG decreases by approximately 0.068 units.
- Statistical significance: p-value = 1.79e-07 (< 0.05) $\rightarrow$ horsepower is a statistically significant predictor of MPG.

Note: As the number of independent variables grows, so does R^2 . Therefore, a more complex model will always have a higher R^2 value. However, model selection has a key principle: parsimony. In simple terms, simple models that perform well should be preferred to complex ones that give little extra information and could lead to overfitting. That is why adjusted R^2 is a more robust metric for model evaluation. It essentially introduces a penalty for extra

variables and increases only when the extra variables actually improve the model. This helps balance complexity, model accuracy and overfitting. There are plenty of other metrics to determine the number of covariates in regression. Some of them are mentioned in Chapter 3.

2.3 Step 3: Hypothesis Testing

For each model, we test:

$$H_0 : \beta_1 = 0 \quad \text{vs} \quad H_1 : \beta_1 \neq 0$$

```
# The p-values are provided in summary()
# Model 1 (wt): p-value = 1.29e-10
# Model 2 (hp): p-value = 1.79e-07
```

It follows that wt is a significant predictor and hp is a significant predictor too.

2.4 Step 4: Confidence Intervals for Coefficients (90%)

```
# 90% confidence intervals for Model 1
confint(m1, level = 0.90)

##                5 %        95 %
## (Intercept) 34.098303 40.471950
## wt          -6.293412 -4.395531

# 90% confidence intervals for Model 2
confint(m2, level = 0.90)

##                5 %        95 %
## (Intercept) 27.32567042 32.87205066
## hp          -0.08540338 -0.05105318
```

Interpretation of Confidence Intervals

For a 90% confidence interval:

- We are 90% confident that the true population slope lies within this interval.
- If the interval does NOT contain 0, we reject H_0 at $\alpha = 0.10$.

2.5 Step 5: Compare R^2 and Adjusted R^2

```
# Extract R-squared from both models
summary(m1)$r.squared      # 0.7528
summary(m1)$adj.r.squared  # 0.7446
```

```
summary(m2)$r.squared      # 0.6024  
summary(m2)$adj.r.squared  # 0.5892
```

Interpretation

- Weight model explains 75.3% of the variation in MPG.
- Horsepower model explains 60.2% of the variation in MPG.
- Weight is a better predictor of MPG than horsepower (higher R^2).

Chapter 3

Fitting a Multiple Linear Regression Model

Now we extend our analysis to include multiple predictors simultaneously.

- Dependent Variable (Y): `mpg` (miles per gallon)
- Independent Variables (X): `wt` (weight), `hp` (horsepower), `cyl` (cylinders), `am` (transmission)

3.1 Step 1: Null Model and Full Model

```
# Null model (intercept only)
m0 <- lm(mpg ~ 1, data = cars)
summary(m0)

##
## Call:
## lm(formula = mpg ~ 1, data = cars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -9.6906 -4.6656 -0.8906  2.7094 13.8094
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   20.091      1.065   18.86  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.027 on 31 degrees of freedom
```

```

# Full model (all candidate predictors)
m <- lm(mpg ~ wt + hp + cyl + am, data = cars)
summary(m)

##
## Call:
## lm(formula = mpg ~ wt + hp + cyl + am, data = cars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.4765 -1.8471 -0.5544  1.2758  5.6608
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 36.14654     3.10478  11.642 4.94e-12 ***
## wt          -2.60648     0.91984   -2.834  0.0086 **
## hp           -0.02495     0.01365   -1.828  0.0786 .
## cyl          -0.74516     0.58279   -1.279  0.2119
## am           1.47805     1.44115    1.026  0.3142
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.509 on 27 degrees of freedom
## Multiple R-squared:  0.849, Adjusted R-squared:  0.8267
## F-statistic: 37.96 on 4 and 27 DF,  p-value: 1.025e-10

```

3.2 Step 2: Model Selection Using Stepwise Regression

We use forward stepwise selection based on AIC (Akaike Information Criterion). Lower AIC indicates a better balance of fit and parsimony.

Forward Selection with AIC (k = 2)

```

step(m0,
      scope = list(lower = formula(m0), upper = formula(m)),
      direction = "forward", k = 2) # AIC

## Start:  AIC=115.94
## mpg ~ 1
##
##      Df Sum of Sq  RSS   AIC
## + wt   1    847.73 278.32  73.217
## + cyl   1    817.71 308.33  76.494
## + hp    1    678.37 447.67  88.427

```

```
## + am      1      405.15   720.90 103.672
## <none>                1126.05 115.943
##
## Step:   AIC=73.22
## mpg ~ wt
##
##           Df Sum of Sq    RSS    AIC
## + cyl     1     87.150 191.17 63.198
## + hp      1     83.274 195.05 63.840
## <none>                278.32 73.217
## + am      1       0.002 278.32 75.217
##
## Step:   AIC=63.2
## mpg ~ wt + cyl
##
##           Df Sum of Sq    RSS    AIC
## + hp      1    14.5514 176.62 62.665
## <none>                191.17 63.198
## + am      1     0.1249 191.05 65.177
##
## Step:   AIC=62.66
## mpg ~ wt + cyl + hp
##
##           Df Sum of Sq    RSS    AIC
## <none>                176.62 62.665
## + am      1     6.6228 170.00 63.442
##
## Call:
## lm(formula = mpg ~ wt + cyl + hp, data = cars)
##
## Coefficients:
## (Intercept)          wt          cyl          hp
##    38.75179    -3.16697   -0.94162   -0.01804
```

Forward Selection with BIC ($k = \log(n)$)

```
step(m0,
      scope = list(lower = formula(m0), upper = formula(m)),
      direction = "forward", k = log(nrow(cars))) # BIC

## Start:   AIC=117.41
## mpg ~ 1
##
##           Df Sum of Sq    RSS    AIC
```

```
## + wt      1      847.73  278.32  76.149
## + cyl      1      817.71  308.33  79.426
## + hp       1      678.37  447.67  91.358
## + am       1      405.15  720.90 106.604
## <none>                1126.05 117.409
##
## Step:   AIC=76.15
## mpg ~ wt
##
##           Df Sum of Sq    RSS    AIC
## + cyl      1      87.150 191.17 67.595
## + hp       1      83.274 195.05 68.237
## <none>                278.32 76.149
## + am       1         0.002 278.32 79.614
##
## Step:   AIC=67.6
## mpg ~ wt + cyl
##
##           Df Sum of Sq    RSS    AIC
## <none>                191.17 67.595
## + hp       1     14.5514 176.62 68.528
## + am       1      0.1249 191.05 71.040
##
## Call:
## lm(formula = mpg ~ wt + cyl, data = cars)
##
## Coefficients:
## (Intercept)          wt          cyl
##      39.686       -3.191       -1.508
```

AIC vs BIC Selection

AIC ($k = 2$): $\text{mpg} \sim \text{wt} + \text{cyl} + \text{hp}$

BIC ($k = \log(n)$): $\text{mpg} \sim \text{wt} + \text{cyl}$

Note: BIC prefers simpler models because it penalizes additional parameters more heavily. BIC tends to be more parsimonious as the number of observations, datapoints grows.

3.3 Model Selection: A Deeper Look

Model selection is one of the most interesting areas in statistics. Unlike hypothesis testing, where the null hypothesis is either rejected or not, model selection forces us to make a choice: which model best balances fit and complexity?

The Parsimony Principle

The principle of parsimony (also known as Occam's razor) states that among competing models that explain the data equally well, the simplest one should be preferred. Why?

- Simpler models are easier to interpret
- They are less likely to overfit the data
- They generalize better to new observations
- They require fewer parameters to estimate

However, a model that is too simple will underfit, missing important patterns. A model that is too complex will overfit, capturing noise as if it were signal. Model selection metrics help us find the sweet spot.

AIC (Akaike Information Criterion)

AIC is defined as:

$$\text{AIC} = -2 \ln(\hat{L}) + 2k$$

where:

- $\hat{L}$ is the maximized likelihood of the model
- k is the number of parameters (including the intercept)

In general, lower AIC indicates a better model. The $2k$ term penalizes additional parameters. This is a vanilla penalty, compared to others. It should be noted that AIC is asymptotically efficient, i.e it minimizes prediction error.

BIC (Bayesian Information Criterion)

BIC is defined as:

$$\text{BIC} = -2 \ln(\hat{L}) + k \ln(n)$$

where:

- $\hat{L}$ is the maximized likelihood
- k is the number of parameters
- n is the sample size

Again, lower BIC indicates a better model. In this case, the penalty term $k \ln(n)$ is larger than AIC's $2k$ when $n > 7$. BIC is consistent as it selects the true model asymptotically. Note that BIC is more parsimonious than AIC as it tends to select simpler models. This is due to the penalty term!

Considerations

1. No single metric is "correct". Different criteria answer different questions.
2. Use domain/ expert knowledge. A model that makes theoretical sense is often better than one that just has a lower AIC.
3. It is advisable to report multiple metrics. For instance, AIC, BIC, and adjusted R^2 , to give a complete picture.

3.4 Step 3: The Best Fitted Model (AIC-based)

Based on the AIC stepwise output, the best model is:

```
best_fit <- lm(mpg ~ wt + cyl + hp, data = cars)
summary(best_fit)

##
## Call:
## lm(formula = mpg ~ wt + cyl + hp, data = cars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.9290 -1.5598 -0.5311  1.1850  5.8986
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  38.75179    1.78686   21.687 < 2e-16 ***
## wt          -3.16697    0.74058   -4.276 0.000199 ***
## cyl          -0.94162    0.55092   -1.709 0.098480 .
## hp           -0.01804    0.01188   -1.519 0.140015
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.512 on 28 degrees of freedom
## Multiple R-squared:  0.8431, Adjusted R-squared:  0.8263
## F-statistic: 50.17 on 3 and 28 DF,  p-value: 2.184e-11
```

3.5 Step 4: Interpreting the Best Model Output

Coefficient Interpretations

Variable	Estimate	Std. Error	p-value	Significant?
Intercept	38.7518	1.7869	$< 2e-16$	Yes
wt (weight)	-3.16697	0.7406	0.000199	Yes
cyl (cylinders)	-0.94162	0.5509	0.0985	No (borderline)
hp (horsepower)	-0.01804	0.0119	0.1400	No

Table 3.1: Coefficient summary for the best fitting model

- Weight (wt): Holding cylinders and horsepower constant, each additional 1000 lbs of weight decreases MPG by approximately 3.17 units. ($p = 0.0002$)
- Cylinders (cyl): Holding weight and horsepower constant, each additional cylinder decreases MPG by approximately 0.94 units. ($p = 0.0985$)
- Horsepower (hp): Holding weight and cylinders constant, each additional horsepower decreases MPG by approximately 0.018 units. ($p = 0.1400$)

Model Fit Statistics

- Multiple R-squared: 0.8431 (84.3% of variance explained)
- Adjusted R-squared: 0.8263 (penalized for number of predictors)
- F-statistic: 50.17 on 3 and 28 DF
- p-value: 2.184e-11 (model is significant overall)

3.6 Step 5: Confidence Intervals for Coefficients (95%)

```
confint(best_fit, level = 0.95)

##              2.5 %          97.5 %
## (Intercept) 35.0915623 42.412012412
## wt          -4.6839740 -1.649972191
## cyl         -2.0701179  0.186884238
## hp          -0.0423655  0.006289293
```

Interpretation of Confidence Intervals

- The 95% CI for **wt** is $[-4.68, -1.65]$. It does not contain 0 $\rightarrow$ weight is a significant predictor.
- The 95% CI for **cyl** is $[-2.07, 0.19]$. It contains 0 $\rightarrow$ cylinders may not be significant at $\alpha = 0.05$.
- The 95% CI for **hp** is $[-0.042, 0.006]$. It contains 0 $\rightarrow$ horsepower is not significant at $\alpha = 0.05$.

3.7 Step 6: Residual Diagnostics

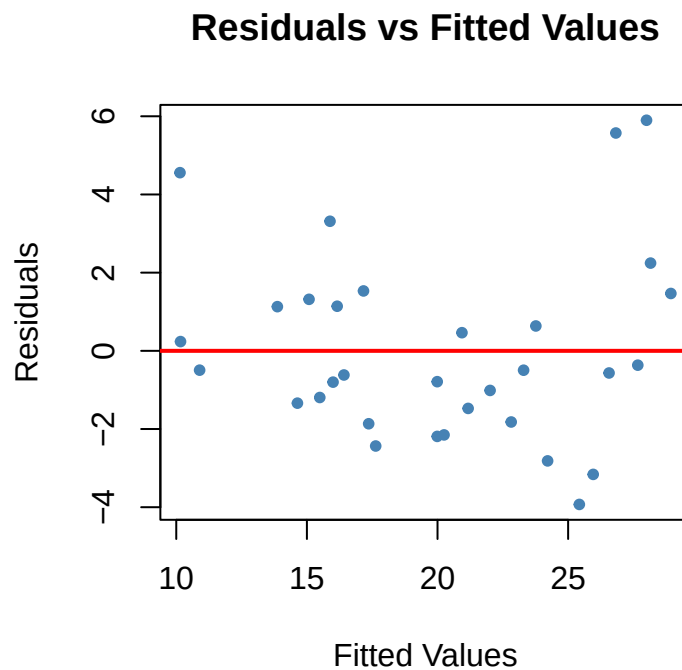
We must check three key assumptions:

1. Independence of errors
2. Homoscedasticity (constant variance of errors)
3. Normality of errors

```
# Extract residuals and fitted values
fitted <- best_fit$fitted.values
res <- best_fit$residuals
```

3.7.1 Homoscedasticity Check

```
# Residuals vs Fitted Values plot
par(mfrow = c(1, 1))
plot(x = fitted, y = res,
     main = "Residuals vs Fitted Values",
     xlab = "Fitted Values",
     ylab = "Residuals",
     pch = 20,
     col = "steelblue")
abline(h = 0, col = "red", lwd = 2)
```



```

# Breusch-Pagan test
lmtest::bptest(best_fit)

##
## studentized Breusch-Pagan test
##
## data: best_fit
## BP = 2.9351, df = 3, p-value = 0.4017

```

Interpretation: $p\text{-value} = 0.4017 > 0.05 \rightarrow$ fail to reject H_0 . There is no evidence of heteroscedasticity. The constant variance assumption holds.

3.7.2 Normality Check

```

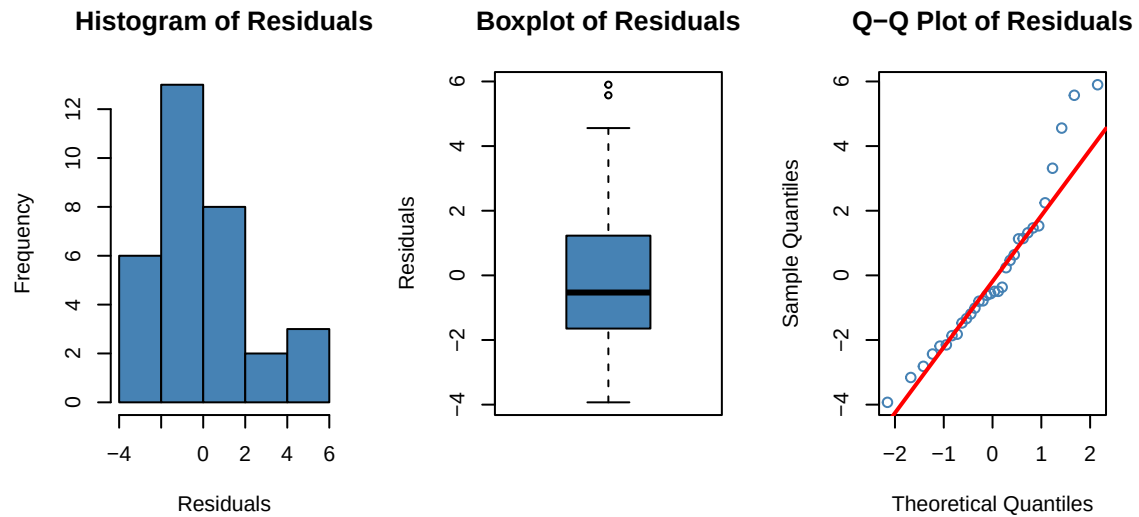
# Diagnostic plots
par(mfrow = c(1, 3))

# Histogram of residuals
hist(res,
      main = "Histogram of Residuals",
      xlab = "Residuals",
      col = "steelblue")

# Boxplot of residuals
boxplot(res,
        main = "Boxplot of Residuals",
        ylab = "Residuals",
        col = "steelblue")

# Q-Q plot
qqnorm(res,
       main = "Q-Q Plot of Residuals",
       col = "steelblue")
qqline(res, col = "red", lwd = 2)

```



```
# Shapiro-Wilk test
shapiro.test(res)

##
##  Shapiro-Wilk normality test
##
## data:  res
## W = 0.93455, p-value = 0.05252
```

Interpretation: $p\text{-value} = 0.05252 > 0.05 \rightarrow$ fail to reject H_0 . The residuals are normally distributed (assumption holds, though borderline).

Chapter 4

Analysis of Variance (ANOVA)

Objective

The objective of Analysis of Variance (ANOVA) is to determine whether there are statistically significant differences among the means of three or more groups. Remember that for two groups we can check with a t-test. ANOVA generalises this idea. In this analysis, we examine the effect of categorical variables on `mpg` (fuel efficiency). Specifically, we investigate:

- Response variable (Y): `mpg` (miles per gallon)
- Factors (X variables):
 - `cyl`: number of cylinders (4, 6, 8), categorical with 3 levels
 - `am`: transmission type (0 = automatic, 1 = manual), categorical with 2 levels

ANOVA Assumptions

Before interpreting the results, we must verify three key assumptions:

1. Independence of observations: the data points are independent (satisfied by experimental design)
2. Normality of residuals: the errors follow a normal distribution
3. Homogeneity of variances: the variances are equal across all groups

4.1 Step 1: Prepare the Data

Ensure categorical variables are properly formatted as factors:

```
# Ensure categorical variables are factors
cars$cyl <- as.factor(cars$cyl)
cars$am <- as.factor(cars$am)
```

4.2 Step 2: Two-Way ANOVA with Interaction

We first fit a two-way ANOVA model including both main effects and their interaction:

```
anov <- aov(mpg ~ cyl * am, data = cars)
summary(anov)

##              Df Sum Sq Mean Sq F value    Pr(>F)    
## cyl           2  824.8   412.4  44.852 3.73e-09 ***
## am            1   36.8    36.8   3.999  0.0561 .    
## cyl:am        2   25.4    12.7   1.383  0.2686    
## Residuals    26  239.1     9.2                     
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The interaction term (`cyl:am`) has $p\text{-value} = 0.2686 > 0.05$. Therefore, the interaction is not statistically significant. This means the effect of transmission type on MPG does not depend on the number of cylinders. We can proceed with an additive model, without interaction.

Note: In order to be able to assume a two-way ANOVA model with interaction it is essential that there are more than one observations/ data points per treatment (i.e combination of levels from both factors). The reason for this is as follows: Suppose we have two factors A and B with two levels a, b each. When there is one observation per treatment (Aa, Ab, Ba, Bb) then there is no within the treatment variance. This means that our observation, let it be x_b^A , in treatment Ab for example could be extremely biased without us ever knowing (we cannot compare against other observations and a treatment mean). Therefore we could mistakenly be led to the conclusion that there is interaction between the factors when there is in fact no interaction and vice versa.

Tip: Always check the number of observations per treatment before assuming an ANOVA model. When there is only one observation in at least one of the treatments, the most complex model that can be assumed is a two-way ANOVA model with no interaction.

4.3 Step 3: Two-Way ANOVA without Interaction

```
anov2 <- aov(mpg ~ cyl + am, data = cars)
summary(anov2)

##              Df Sum Sq Mean Sq F value    Pr(>F)    
## cyl           2  824.8   412.4  43.657 2.48e-09 ***
## am            1   36.8    36.8   3.892  0.0585 .    
## Residuals    28  264.5     9.4                     
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- cyl (number of cylinders): $p\text{-value} < 0.001 \rightarrow$ **highly significant**

- `am` (transmission type): $p\text{-value} = 0.0585 > 0.05 \rightarrow$ not statistically significant (borderline)
- We remove the non-significant factor `am` and proceed with a one-way ANOVA.

4.4 Step 4: One-Way ANOVA

```
anov3 <- aov(mpg ~ cyl, data = cars)
summary(anov3)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## cyl              2   824.8    412.4     39.7 4.98e-09 ***
## Residuals       29   301.3     10.4
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

There is a highly significant difference in MPG among cars with different numbers of cylinders ($p = 4.98e-09$). However, the F-test only tells us that at least one group differs. It does not specify which groups differ. To identify which cylinder groups differ, we perform a post-hoc test.

4.5 Step 5: Post-Hoc Comparisons (Tukey HSD)

```
TukeyHSD(anov3)
```

```
##      Tukey multiple comparisons of means
##      95% family-wise confidence level
##
## Fit: aov(formula = mpg ~ cyl, data = cars)
##
## $cyl
##              diff              lwr              upr              p adj
## 6-4    -6.920779   -10.769350   -3.0722086   0.0003424
## 8-4   -11.563636   -14.770779   -8.3564942   0.0000000
## 8-6    -4.642857    -8.327583   -0.9581313   0.0112287
```

All pairwise comparisons are statistically significant ($p < 0.05$). Cars with fewer cylinders have significantly higher MPG. The difference is largest between 8-cylinder and 4-cylinder cars (11.56 MPG lower).

4.6 Step 6: Residual Diagnostics

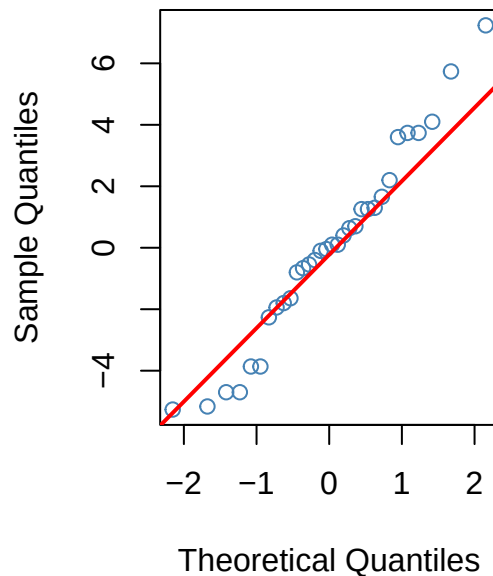
Normality Check

```

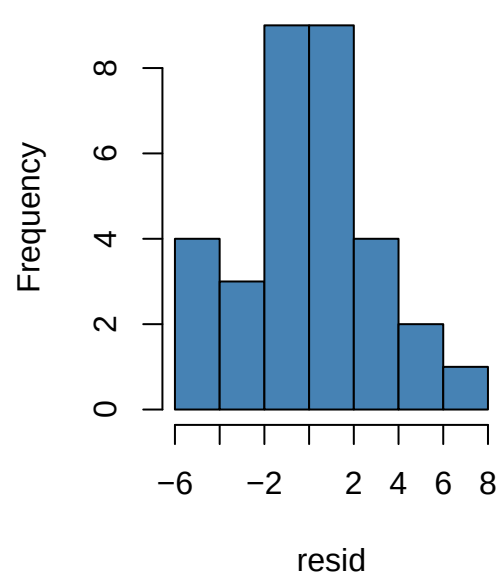
resid <- anov3$residuals
par(mfrow = c(1, 2))
qqnorm(resid, main = "Q-Q Plot of Residuals", col = "steelblue")
qqline(resid, col = "red", lwd = 2)
hist(resid, main = "Histogram of Residuals", col = "steelblue")

```

Q-Q Plot of Residuals



Histogram of Residuals



```

shapiro.test(resid)

##
##  Shapiro-Wilk normality test
##
## data:  resid
## W = 0.97065, p-value = 0.5177

```

- The Q-Q plot shows points approximately following the reference line.
- The Shapiro-Wilk test yields $p\text{-value} = 0.5177 > 0.05$.
- We fail to reject the null hypothesis of normality.
- Conclusion: The normality assumption is satisfied.

Homogeneity of Variances Check

```
bartlett.test(mpg ~ cyl, data = cars)

##
## Bartlett test of homogeneity of variances
##
## data:  mpg by cyl
## Bartlett's K-squared = 8.3934, df = 2, p-value = 0.01505
```

- The Bartlett test yields $p\text{-value} = 0.01505 < 0.05$.
- We reject the null hypothesis of equal variances.
- Conclusion: The homogeneity of variances assumption is violated. Since the homogeneity assumption is violated, we may consider alternative approaches but this is beyond the scope of this report.

Note to reader

Hopefully, this guide has provided a useful introduction to the basic methods and interpretational principles of linear regression in R. A wide variety of built-in R datasets can be used to further practice these techniques. Some particularly useful examples include `iris`, `women`, and `Pima.tr`.